Generative AI

Notes

Author:
Arun S
Data Scientist
arunkumars.co.in.

Embeddings in NLP

Embeddings are dense, low-dimensional vector representations of text (words, sentences, or documents) that encode semantic and syntactic meaning by mapping discrete symbols into a continuous vector space.

Instead of treating words as independent tokens, embeddings allow NLP models to capture similarity, context, and relationships through geometric structure, enabling generalization, semantic search, and reasoning.

Modern NLP systems rely on learned embeddings as the foundational representation layer before any higher-level modelling.

a. Embeddings transform discrete language into a continuous space where semantic similarity corresponds to vector proximity
b. Contextual embeddings adapt representations based on surrounding words, unlike static embeddings
c. Most NLP tasks reduce to learning or comparing embeddings effectively

Similarity between two embeddings is commonly measured using cosine similarity. Embedding matrices are learned parameters optimized through downstream objectives

Tokenization

Tokenization is the process of converting raw text into smaller units called **tokens** (words, subwords, or characters) that a language model can process.

Since neural models operate on numerical inputs, tokenization acts as the **interface between human language and vector-based computation**, directly influencing vocabulary size, model efficiency, and downstream performance. Modern NLP relies on **subword tokenization** to balance vocabulary coverage, semantic consistency, and robustness to rare or unseen words.

a. Tokenization determines how text is segmented and directly impacts model accuracy, efficiency, and generalization
b. Sub word tokenization mitigates the out-of-vocabulary problem by decomposing rare words into frequent units
c. Token boundaries define what the model can and cannot represent explicitly

Stemming and Lemmatization

Stemming and lemmatization normalize words to a base form to reduce sparsity and improve lexical matching. Stemming applies heuristic rules, while lemmatization uses linguistic knowledge to recover canonical forms.

Stop words

Stop words are high-frequency function words that often carry limited semantic content for certain tasks. Removing them can simplify representations and improve efficiency in classical NLP pipelines.

Language Modelling

Language modeling estimates the probability distribution over sequences of tokens, enabling prediction of the next token given prior context. It is the core objective behind generative NLP systems and underpins pre-training of modern LLMs.

Named Entity Recognition (NER)

Named Entity Recognition is an NLP task that identifies and classifies spans of text into predefined **entity categories** such as person, organization, location, date, or quantity.

NER converts unstructured text into structured information, enabling downstream tasks like information extraction, search, question answering, and knowledge graph construction.

Modern NER systems rely on contextual embeddings and sequence modelling to capture both semantic meaning and token-level dependencies.

Autoencoder

An **Autoencoder** is a neural network designed to learn a **compressed representation of data** by training the model to **reconstruct its input**.

It consists of two main components: an **Encoder**, which maps the input to a lower-dimensional latent space, and a **Decoder**, which reconstructs the input from this latent representation.

Autoencoders are primarily used for **representation learning**, not prediction.

The architecture consists of:

a. Input layer
b. Encoder network
c. Latent (bottleneck) representation
d. Decoder network
e. Reconstructed output

Encoder - Encoder transforms the input data into a latent representation.

Latent Space (Bottleneck) - The **latent space** is a compressed representation of the input.

Decoder - Decoder reconstructs the original input from the latent representation.

Autoencoders are trained to minimize reconstruction error**.**

Applications of Autoencoders:

a. Dimensionality reduction
b. Feature learning
c. Data compression
d. Anomaly detection
e. Noise removal
f. Pretraining deep networks

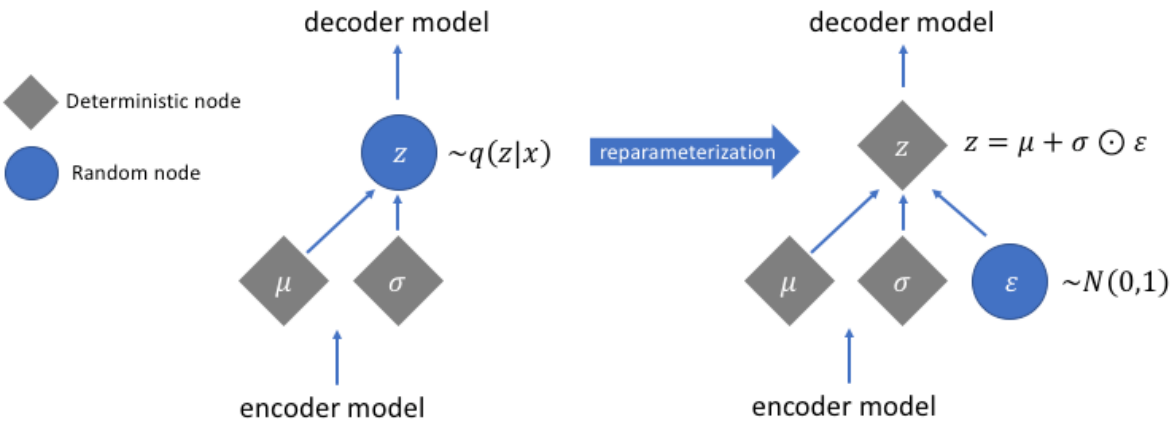
Variational Autoencoder

Variational Autoencoder (VAE) is a probabilistic generative model that learns a latent variable distribution from data and can generate new samples by sampling from this distribution. Unlike a standard autoencoder, which learns a deterministic latent representation, a VAE learns a distribution over latent variables, enabling principled data generation.

VAEs combine neural networks with probabilistic modeling and variational inference.

Encoder (Inference Network) - The encoder does not output a single latent vector. Instead, it outputs **parameters of a probability distribution**.

Latent Variable Sampling - Latent variables are sampled from the learned distribution



Decoder (Generative Network) - The decoder reconstructs data from sampled latent variables. It defines the likelihood of the data given the latent code. The decoder learns how latent variables map to observable data.

Aspect	*Autoencoder*	*VAE*
Latent space	Deterministic	Probabilistic
Generative	No	Yes
Objective	Reconstruction	ELBO
Sampling	Not meaningful	Meaningful
Regularization	Optional	Built-in

Generative Adversarial Networks (GAN)

A Generative Adversarial Network (GAN) is a generative modeling framework that learns to produce realistic data by training two neural networks in competition: a Generator, which synthesizes data, and a Discriminator, which evaluates whether data is real or fake.

Through this adversarial process, the generator learns the underlying data distribution and produces increasingly realistic samples.

GANs do not explicitly model likelihoods; they learn via adversarial training.

A GAN consists of:

a. Generator
b. Discriminator
c. Noise prior
d. Adversarial loss function

Generator - **Generator** maps random noise to data space and attempts to produce realistic samples.
Discriminator - Discriminator is a binary classifier that distinguishes real data from generated data.

Characteristics:

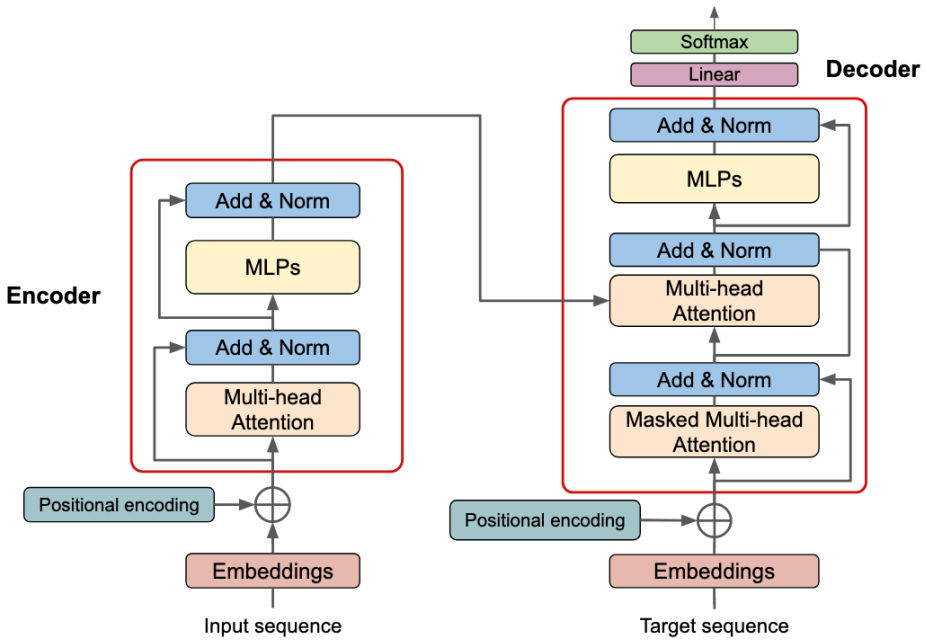
a. Trained with supervised signals (real vs fake)
b. Provides gradients to guide the generator
c. Must remain balanced with the generator

Transformer Architecture

The Transformer is a deep learning architecture designed to model sequential data using attention mechanisms instead of recurrence or convolution. It processes entire sequences in parallel and captures long-range dependencies through self-attention.

The Transformer architecture was introduced to overcome the limitations of RNNs and CNNs in sequence modeling, especially issues related to scalability, long-term dependencies, and training efficiency.

Transformers are the backbone of - Large Language Models, Vision Transformers, Multimodal models, Foundation models.



The Transformer consists of:

a. An Encoder stack
b. A Decoder stack
c. Embedding and positional encoding layers
d. Output projection layers

Token Embeddings - Input tokens are mapped to dense vectors of fixed dimension. These embeddings capture semantic meaning but contain **no positional information**.

.

Positional Encoding - Positional Encoding is added to token embeddings to inject sequence order. Without positional encoding, attention is permutation invariant.

Encoder Stack - The encoder consists of multiple identical layers stacked vertically. Each encoder layer contains - Multi-Head Self-Attention, Feed-Forward Neural Network, Residual connections, Layer Normalization

Self-Attention in Encoder - Each token attends to **all other tokens** in the input sequence. It enables Global context awareness, Long-range dependency modelling, Contextualized embeddings

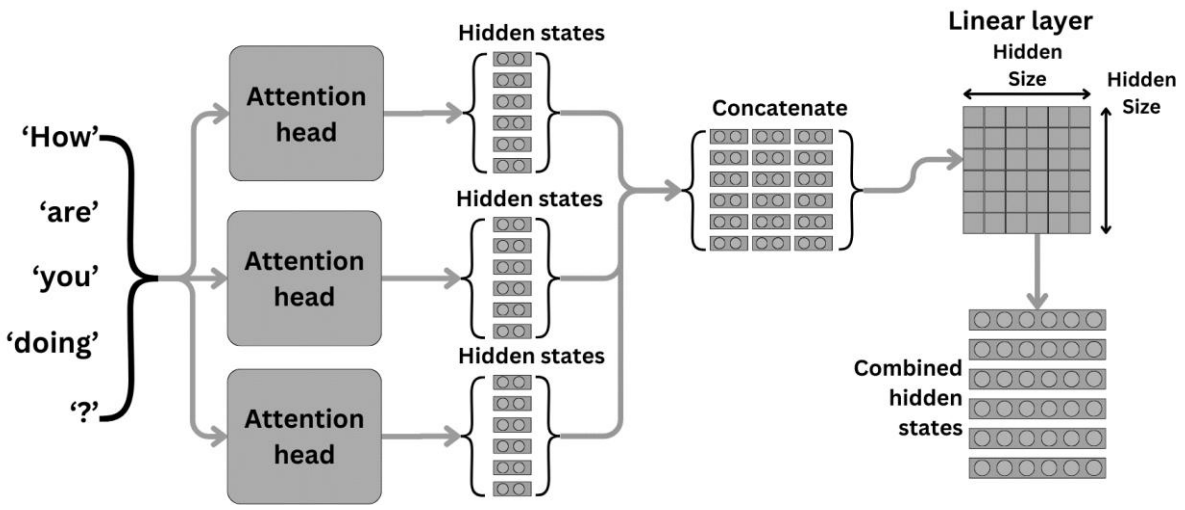
Decoder Stack – Decoder consists of repeated identical layers. Each decoder layer contains: Masked Multi-Head Self-Attention, Cross-Attention (Encoder–Decoder Attention), Feed-Forward Network, Residual connections, Layer Normalization

Masked Self-Attention - The decoder prevents access to future tokens using a causal mask. This ensures autoregressive generation.

Cross-Attention - Cross-attention allows the decoder to attend to encoder outputs. Queries come from decoder, Keys and values come from encoder. This enables alignment between input and output sequences.

Multi-Head Attention - Multi-Head Attention allows the model to attend to information from multiple representation subspaces simultaneously. Instead of a single attention operation, multiple heads operate in parallel.

- Captures syntactic relations
- Captures semantic relations
- Improves expressiveness
- Prevents attention collapse



Feed-Forward Network (FFN) - Each Transformer layer contains a position-wise feed-forward network. It is applied independently to each token, Same parameters across all positions and introduces non-linearity

The FFN expands and contracts dimensionality, increasing model capacity.

Residual Connections - Add input of a sublayer to its output. It enables deep stacking of layers and improves gradient flow.

Layer Normalization - Layer Normalization stabilizes activations at each layer. Transformers rely on Layer Normalization rather than Batch Normalization.

Output Layer - The decoder output is projected to vocabulary space using a linear layer followed by soft max. It produces Probability distribution over tokens and next-token predictions during generation**.**

Transformer Variants

Encoder-Only

a. Used for representation learning. Examples: BERT
b. Suitable for classification and understanding tasks

Decoder-Only

- Used for generation. Examples: GPT
- Autoregressive by design

Encoder–Decoder

a. Used for sequence-to-sequence tasks. Examples: T5
b. Strong alignment capability

Strengths of Transformer Architecture

a. Parallelizable
b. Scales with data and compute
c. Captures long-range dependencies
d. Flexible across modalities

Batch Normalization

Batch Normalization is a technique used in deep neural networks to normalize intermediate layer activations during training. It standardizes the activations of each layer for a mini-batch to have zero mean and unit variance, followed by a learnable affine transformation.

Batch Normalization stabilizes training by reducing undesirable variations in the distribution of activations as data flows through the network.

It is typically applied after linear transformation (Dense or Convolution) and before the non-linear activation (ReLU, GELU)

As networks become deeper:

a. Small changes in parameters can cause large shifts in activation distributions
b. Training becomes unstable
c. Learning slows down due to poor gradient flow

Batch Normalization addresses this by:

a. Reducing internal covariate shift
b. Improving gradient propagation
c. Allowing higher learning rates
d. Acting as a regularizes

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Limitations of Batch Normalization

a. Performance degrades with very small batch sizes
b. Batch dependency can cause instability in distributed training
c. Less effective for recurrent and autoregressive models
d. Not ideal for online or streaming inference

Layer Normalization

Layer Normalization is a technique used in deep neural networks to normalize intermediate layer activations during training. It standardizes the activations within a single data sample across the feature dimension to have zero mean and unit variance, followed by a learnable affine transformation.

Layer Normalization stabilizes training by reducing variations in activation distributions independent of batch size, making it especially suitable for sequential and autoregressive models.

It is typically applied **after the linear transformation** (Dense or Projection layer) and **before the non-linear activation** (ReLU, GELU).

Layer Normalization addresses this by:

a. Normalizing activations at the sample level
b. Eliminating dependency on mini-batch statistics
c. Stabilizing hidden state dynamics
d. Enabling consistent behavior during training and inference

Layer Normalization is the default normalization technique in:

a. Transformers
b. Large Language Models
c. RNNs and LSTMs
d. Autoregressive and online inference systems

Batch Normalization vs Layer Normalization

Batch Normalization and Layer Normalization are techniques used in deep neural networks to **stabilize and accelerate training** by normalizing intermediate activations.
The key difference lies in **what is normalized and along which dimension**.

Training vs Inference Behavior

Batch Normalization

a. Uses batch statistics during training
b. Uses running averages during inference
c. Output depends on other samples during training

Layer Normalization

a. Uses per-sample statistics always
b. No running averages required
c. Identical behavior in training and inference

Aspect	*Batch Normalization*	*Layer Normalization*
Normalization axis	Batch dimension	Feature dimension
Batch-size dependency	Yes	No
Training vs inference	Different	Same
Works with batch size 1	No	Yes
Regularization effect	Strong	Weak
CNN suitability	High	Moderate
Transformer suitability	Low	High

- Batch Normalization is always better than Layer Normalization
- Layer Normalization removes the need for regularization
- Both behave the same during inference
- Normalization removes representational power

Pre-Norm and Post-Norm Transformer Blocks

In **Post-Norm Transformers**, Layer Normalization is applied **after** the residual connection. This was the original design used in the *Attention Is All You Need* paper.

This pattern is applied to both to Self-Attention sublayer and Feed-Forward sublayer

The model first Applies the transformation, adds residual information, Normalizes the combined signal

In **Pre-Norm Transformers**, Layer Normalization is applied **before** the sublayer, and the residual connection bypasses normalization. This design became standard for **deep and large-scale Transformers**.

- Normalizes inputs before transformation
- Preserves a clean residual path
- Allows gradients to flow directly through identity connections

In Pre-Norm, the gradient has a **direct identity path**.

Model Type	*Normalization*
Original Transformer	Post-Norm
BERT (early versions)	Post-Norm
GPT family	Pre-Norm
T5	Pre-Norm
Modern LLMs	Pre-Norm

Attention Mechanism

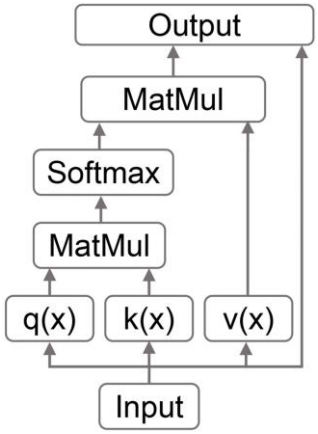
Attention is a mechanism that allows a model to selectively focus on relevant parts of the input while processing information. Instead of treating all inputs equally, attention assigns importance weights to different elements based on their relevance to the current computation.

Attention enables models to:

a. Handle long-range dependencies
b. Avoid information bottlenecks
c. Dynamically route information

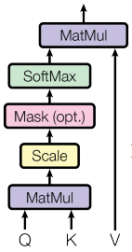
Before attention:

a. RNNs compressed entire sequences into a single hidden state
b. Long-term dependencies degraded
c. Parallelization was impossible
d. Attention removed the need for fixed-size compression and enabled direct access to all tokens at every step.



Scaled Dot-Product Attention

Scaled Dot-Product Attention computes attention weights using the dot product between queries and keys, scaled by the square root of the key dimension to stabilize gradients.



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Without scaling:

a. Dot products grow large in high dimensions
b. Soft max saturates
c. Gradients vanish
d. Scaling ensures stable training and smoother probability distributions.
e.

Self-Attention

Self-Attention allows each token in a sequence to attend to all other tokens in the same sequence, including itself.

All queries, keys, and values come from the same input embedding matrix. This makes self-attention content-driven rather than position-driven.

This enables:

- Global context modelling
- Long-range dependency capture
- Context-aware embeddings

Cross-Attention

Cross-Attention allows queries from one sequence to attend to keys and values from another sequence. It is used in Encoder–decoder architectures, Multimodal models

Use Cases

a. Machine Translation
b. Text-to-image models
c. Speech-to-text systems

Multi-Head Attention

Multi-Head Attention runs multiple attention mechanisms in parallel, each learning a different representation subspace.

Instead of one large attention, attention is factorized into smaller subspaces, improving expressiveness without quadratic parameter growth.

Multiple heads capture:

a. Syntax
b. Semantics
c. Coreference
d. Positional relationships

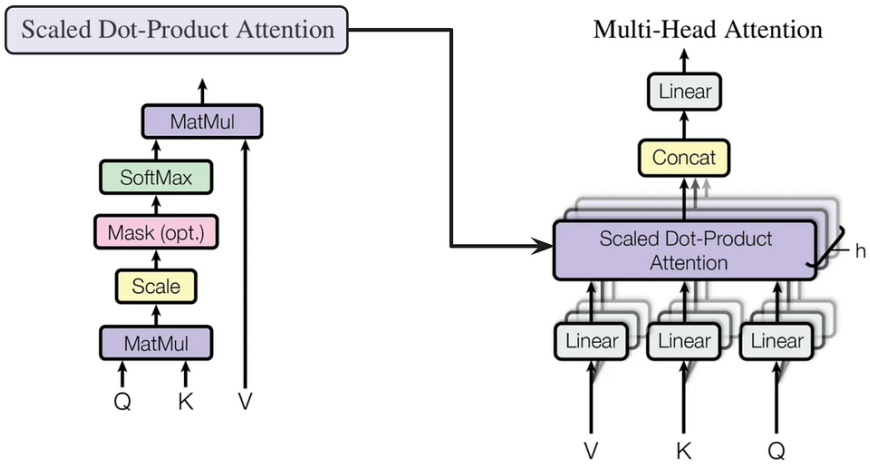


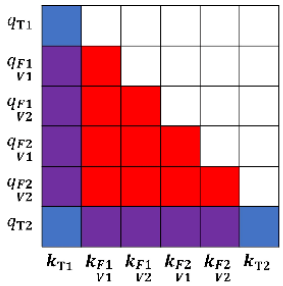
Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Causal (Masked) Attention

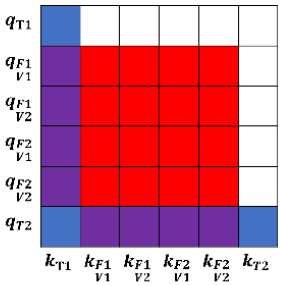
Causal Attention prevents a token from attending to future tokens by applying a mask to the attention matrix. It is used in GPT-style models, Language generation, Time-series forecasting

In autoregressive generation:

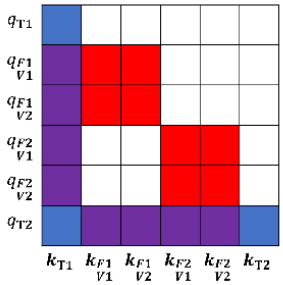
a. The future must be unknown
b. Information leakage must be prevented



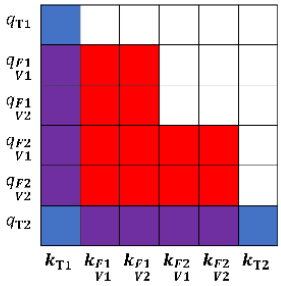
1. Causal Mask



2. Full Visual Mask



3. Fw Block Mask



4. Fw Block Causal Mask

Positional Encoding

Positional Encoding is a technique used in Transformer models to inject order information (position of tokens in a sequence) into the model. Since **self-attention is permutation invariant**, Transformers **cannot know token order by default**.
Positional Encoding fixes this.

Types of Positional Encoding

A. Sinusoidal Positional Encoding (Original Transformer)

I. Fixed
II. No learned parameters
III. Allows extrapolation to longer sequences

B. Learned Positional Embeddings

I. Position is trainable vector
II. Used in BERT, GPT
III. Limited to max sequence length

C. Relative Positional Encoding

I. Encodes distance between tokens
II. Used in Transformer-XL, T5
III. Better for long-context modeling

Sinusoidal Positional Encoding:

For each position pos and dimension i:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

Here, d_{model} is the dimension of the model.

Different frequencies encode short-term & long-term dependencies. Any position can be expressed as a linear function of others. Enables attention heads to learn relative positions implicitly.

Pre – Training

Pre-training is the process of training a model on a large, generic dataset to learn general-purpose representations, before adapting it to a specific downstream task.
The goal is not task performance initially, but representation learning. In modern Deep Learning and Generative AI, pre-training is the foundation step that enables: Data efficiency, Faster convergence, Better generalization.

Types of Pre-Training

Supervised Pre-Training

a. Uses labeled datasets
b. Example: ImageNet pre-training for vision models
c. Transfers task-relevant features

Self-Supervised Pre-Training

a. No manual labels
b. Targets generated from data itself
c. Dominant approach in GenAI

Examples:
a. Masked Language Modeling (BERT)
b. Next Token Prediction (GPT)
c. Contrastive Learning (SimCLR, CLIP)

Unsupervised Pre-Training
a. Learns data distribution directly
b. Example: Autoencoders, language modeling.

Masked Language Modeling (MLM)

Masked Language Modeling is a self-supervised pre-training objective where a model learns to predict masked tokens in a sequence using both left and right context. A fixed percentage of tokens in the input are replaced with a special [MASK] symbol (or corrupted), and the model is trained to recover the original tokens.

a. Uses bidirectional context
b. Commonly used in encoder-based models
c. Enables deep contextual understanding
d. Not naturally generative
e. Learns strong token-level representations

Next Token Prediction (NTP)

Next Token Prediction is an **autoregressive pre-training objective** where a model learns to predict the **next token** in a sequence given all previous tokens. The model is trained using **causal (uni-directional) attention**, making it naturally suited for text generation.

a. Uses left-to-right context only
b. Commonly used in decoder-only models
c. Naturally generative
d. Scales well with data and model size
e. Enables in-context learning and few-shot behavior

Fine-Tuning

Fine-tuning is the process of adapting a pre-trained model to a specific task or domain by continuing training on a smaller, task-relevant dataset. The model starts with learned general representations and adjusts its parameters to optimize performance for a targeted objective.

Fine-tuning leverages prior knowledge acquired during pre-training and focuses on specialization rather than learning from scratch.

Fine-Tuning Workflow

A pre-trained model is loaded with frozen or partially trainable layers.
Task-specific data is introduced.
Loss is computed based on the downstream objective.
Gradients update selected parameters.

This process can range from full-model training to lightweight adaptation methods.

Types of Fine-Tuning

Full Fine-Tuning

a. All parameters are updated
b. Highest flexibility
c. High risk of overfitting
d. Computationally expensive

Partial Fine-Tuning

a. Early layers frozen
b. Later layers updated
c. Preserves general representations

Parameter-Efficient Fine-Tuning (PEFT)

a. Only a small subset of parameters trained
b. Examples: adapters, LoRA, prefix tuning
c. Widely used for LLMs

Weight Pruning and Neuron Pruning

Pruning is a model compression technique that reduces model size and computation by removing parameters or structures that contribute little to the final prediction.

Weight pruning removes individual connections with small importance, while neuron pruning removes entire units (neurons or channels), leading to more structured sparsity and potential speedups on hardware.

a. Weight Pruning zeroes out low-magnitude weights, creating sparse matrices with minimal accuracy loss
b. Neuron Pruning removes whole neurons or channels, yielding structured compression and real inference speed gains
c. Pruning trades redundancy for efficiency and often requires fine-tuning to recover performance

Knowledge Distillation

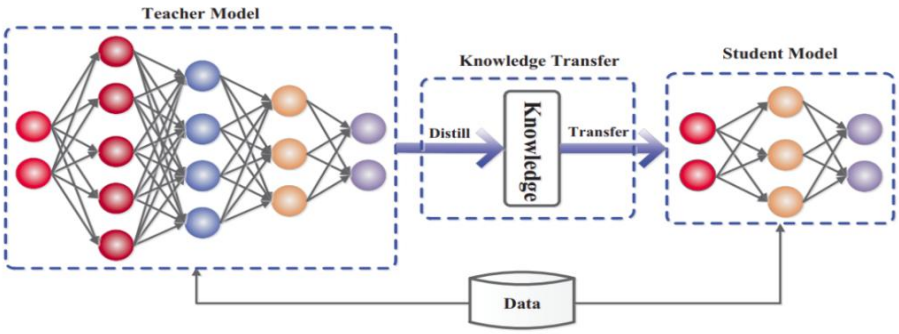
Knowledge Distillation (KD) is a model compression and transfer technique where a large, powerful model (Teacher) transfers its learned knowledge to a smaller, efficient model (Student).

Instead of learning solely from ground-truth labels, the student learns from the soft predictions of the teacher, which encode richer information about class relationships and decision boundaries.

Knowledge distillation enables deploying lightweight models while retaining much of the performance of large models.

KD Working

A teacher model does not only predict the correct answer, but also provides **relative confidence over all classes**.
These soft probabilities contain **dark knowledge** about similarities between classes that hard labels do not reveal.



Parameter-Efficient Fine-Tuning (PEFT) Techniques

Parameter-Efficient Fine-Tuning (PEFT) refers to a family of methods that adapt large pre-trained models to new tasks by training only a small subset of parameters, while keeping the original model weights frozen.

PEFT drastically reduces memory, compute, and storage requirements, making it feasible to customize large language models for multiple tasks or domains without full fine-tuning. These techniques exploit the observation that pre-trained models already contain rich representations and require only lightweight adjustments for specialization.

Types of Parameter-Efficient Fine-Tuning (PEFT) Techniques

Adapters: Small bottleneck layers inserted between Transformer layers; only adapter weights are trained

LoRA (Low-Rank Adaptation): Decomposes weight updates into low-rank matrices, updating only a small subspace

Prefix / Prompt Tuning: Prepends trainable vectors to attention layers or inputs without changing model weights

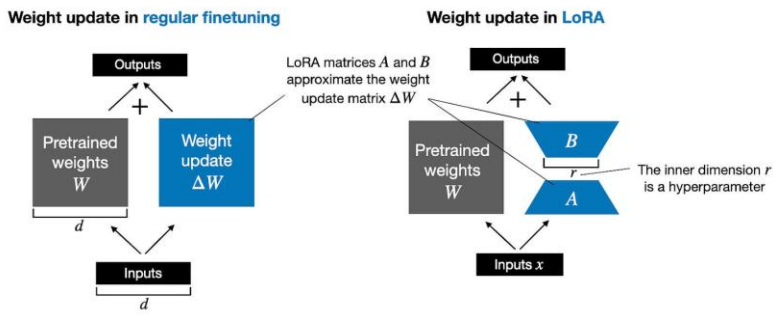
Original weights remain frozen, Trainable parameters lie in a low-dimensional subspace and task adaptation is achieved through additive or injected transformations

LoRA (Low-Rank Adaptation)

LoRA (Low-Rank Adaptation) is a Parameter-Efficient Fine-Tuning technique that adapts large pre-trained models by injecting **low-rank trainable matrices** into existing weight matrices while keeping the original weights **frozen**.

Instead of updating a full weight matrix, LoRA learns a low-dimensional update that captures task-specific changes efficiently, making it highly scalable for large language models.

a. Freezes original model weights and learns only low-rank updates
b. Dramatically reduces trainable parameters and memory usage
c. Enables fast task switching by loading lightweight LoRA adapters



Pre - Training vs Fine Tuning

Pre-training and fine-tuning are two sequential but conceptually different phases in modern deep learning workflows.
Pre-training focuses on learning general representations from large-scale data, while fine-tuning adapts those representations to specific tasks or domains.

Together, they form the backbone of foundation models and large language models.

Pre-training builds knowledge. Fine-tuning applies and aligns that knowledge.

During Pre-Training

a. Poor data quality leads to weak representations
b. Objective misalignment limits downstream usefulness

During Fine-Tuning

a. Overfitting on small datasets
b. Catastrophic forgetting
c. Behavioral over-specialization

Aspect	Pre-Training	Fine-Tuning
Goal	Learn general representations	Specialize for a task
Data size	Very large	Small to medium
Data type	Generic	Task or domain specific
Compute cost	Extremely high	Moderate to low
Learning rate	Relatively higher	Very low
Risk	Underfitting	Overfitting, forgetting
Knowledge gained	Broad	Narrow and targeted

BERT - Bidirectional Encoder Representations from Transformers

BERT is a Transformer-based language model designed to learn deep bidirectional contextual representations of text. Unlike autoregressive models that read text left-to-right, BERT jointly conditions on both left and right context, enabling richer understanding of language.

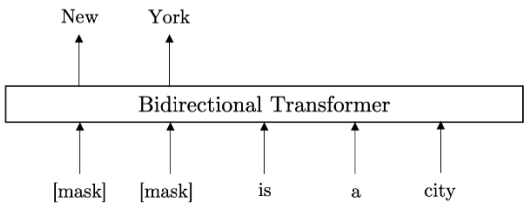
BERT is an encoder-only Transformer trained using self-supervised objectives and serves as a foundation model for language understanding tasks.

BERT consists of:

a. Token embedding layer
b. Segment embedding layer
c. Positional embedding layer
d. Stack of Transformer encoder blocks
e. Task-specific output heads (during fine-tuning)

Strengths of BERT

a. Strong language understanding
b. Excellent transfer learning
c. Simple fine-tuning
d. High performance on NLP benchmarks



$$\log p(\bar{\mathbf{x}}|\hat{\mathbf{x}}) = \sum_{t=1}^{T} \mathrm{mask}_t \log p(x_t|\hat{\mathbf{x}})$$

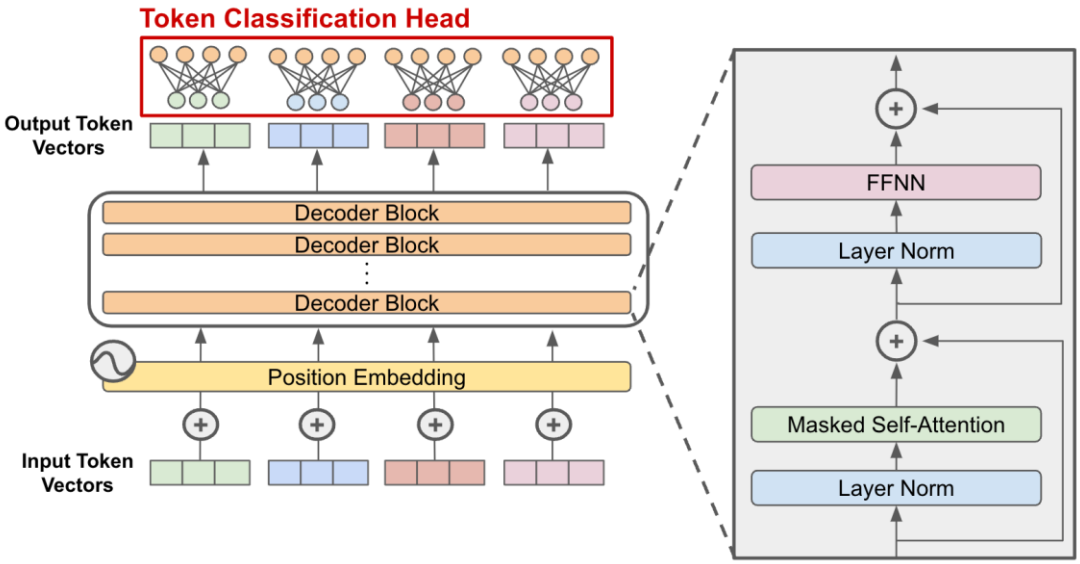
GPT - Generative Pre-trained Transformer

GPT is a **decoder-only Transformer architecture** designed for **autoregressive language modeling**, where text is generated one token at a time by predicting the most probable next token given all previous tokens.

GPT models are trained at scale using self-supervised learning and serve as **general-purpose generative foundation models** capable of reasoning, summarization, translation, code generation, and dialogue.

GPT is built on the principle that **next-token prediction over massive data** is sufficient to learn:

a. Language structure
b. World knowledge
c. Reasoning patterns
d. Task generalization



GPT consists of:

a. Token embedding layer
b. Positional encoding
c. Stack of decoder Transformer blocks
d. Causal self-attention
e. Feed-forward networks
f. Output projection to vocabulary space

Strengths of GPT:

a. Natural language generation
b. Long-form coherent text
c. Reasoning and planning
d. Code generation

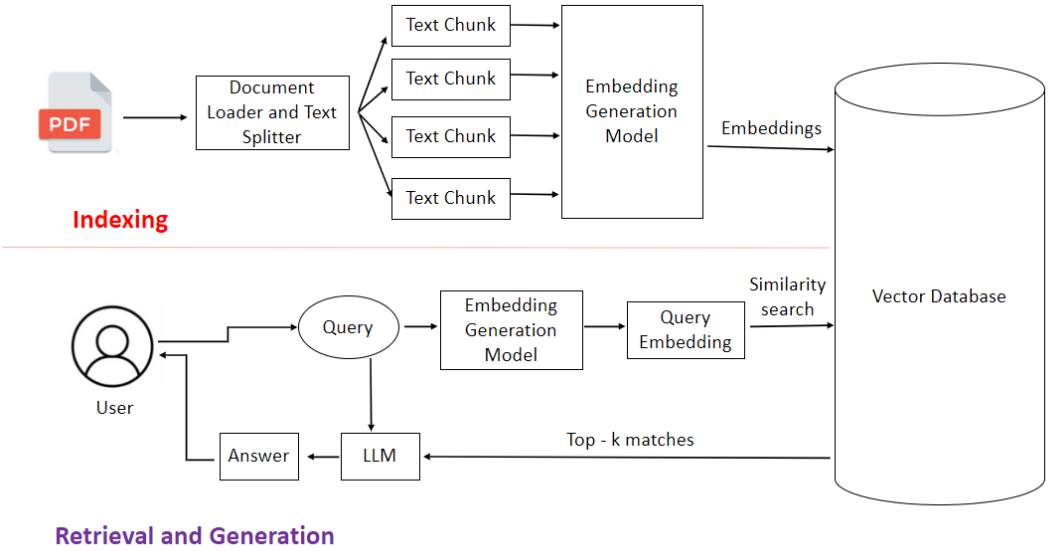
e. Few-shot and zero-shot learning
f. Strong generalization across tasks

Aspect	*GPT*	*BERT*
Architecture	Decoder-only	Encoder-only
Context	Left-to-right	Bidirectional
Objective	Next token prediction	Masked LM
Generation	Yes	No
Best for	Generation	Understanding

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a framework that combines information retrieval with text generation. Instead of relying solely on a model's internal parameters, RAG retrieves relevant external documents at inference time and conditions the generation on this retrieved context.

RAG addresses the core limitation of language models: parametric knowledge is finite, static, and imperfect.



RAG enables:

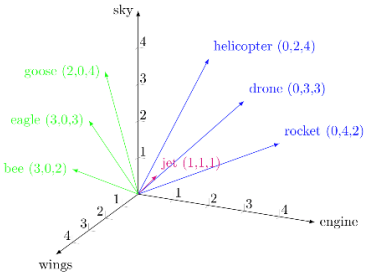
a. Private knowledge assistants
b. Search-augmented chatbots
c. Compliance-aware AI
d. Domain-specific co pilots

RAG Pipeline Overview

Document Ingestion - Documents are collected from PDFs, Databases, Websites, Internal knowledge bases. They are split into smaller **chunks** to enable fine-grained retrieval.

Chunking Strategy - Chunking determines retrieval quality. Poor chunking leads to poor retrieval, regardless of model quality.

Embedding Objective - Text similarity is measured by distance in vector space.



$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Similarity Search - The retriever returns chunks most semantically similar to the query.
Vector Database - The vector database stores embeddings, Enables fast similarity search, Retrieves top-k relevant chunks

Prompt Construction - A typical RAG prompt includes System instructions, Retrieved documents, User query. The quality of prompt formatting heavily influences output quality.

Aspect	*RAG*	*Fine-Tuning*
Knowledge source	External	Internal (weights)
Data freshness	High	Static
Cost	Low	High
Hallucination risk	Lower	Higher
Update speed	Instant	Slow